



Università degli Studi di Padova
Corso di Laurea in Informatica
A. A. 2025/26



Relazione

Concorso Accattivante Accessibile

Studenti	Sofia De Blasi (referente) - 2111014 - sofia.deblasi@studenti.unipd.it
	Nicola Simionato - 2113190 - nicola.simionato.5@studenti.unipd.it
	Nenad Radulovic - 2101059 - nenad.radulovic@studenti.unipd.it
	Giacomo Speggiorin - 2116438 - giacomo.speggiorin@studenti.unipd.it
Indirizzo del sito	https://caa.studenti.math.unipd.it/sdeblasi/
Utente	Username: user, Password: user

1 Introduzione

Il progetto *Student Space* è stato realizzato con l'obiettivo di dimostrare che un sito web può essere al tempo stesso accessibile, gradevole e funzionale, in linea con lo spirito del concorso *Accattivante Accessibile*. La piattaforma raccoglie annunci utili alla vita universitaria (eventi, affitti, ripetizioni, esperimenti) e permette agli utenti di consultarli, filtrarli e pubblicarli in modo semplice e intuitivo.

Fin dalle prime fasi di progettazione, il lavoro è stato guidato dai principi delle WCAG 2.1 livello AA, adottando un approccio orientato alla fruibilità per tutte le categorie di utenti. Ogni scelta relativa a struttura, contenuti, interazioni e design è stata valutata in funzione della sua accessibilità, con particolare attenzione alla semantica HTML, alla navigazione da tastiera, alla gestione dei testi alternativi, ai contrasti cromatici e alla prevedibilità dei comportamenti dell'interfaccia.

L'obiettivo finale è stato quello di creare un sito che non richieda all'utente di adattarsi, ma che si adatti alle sue esigenze.

2 Accessibilità

2.1 Strumenti di verifica

Sono stati utilizzati diversi strumenti di verifica:

- **NVDA**: per testare la gerarchia dei titoli, la lista dei link e dei punti di riferimento, la navigazione tramite landmark e l'annuncio dei messaggi di errore nei form, sia con JavaScript attivo che disattivato.
- **Contrast Checker** (<https://contrastchecker.com/>): per verificare il rispetto del rapporto di contrasto $\geq 4.5:1$.
- **Lighthouse, Silktide e strumenti di ispezione (F12)**: per valutazioni automatiche e test manuali su navigazione da tastiera, accessibilità dei link, dimensioni degli elementi cliccabili e altri criteri WCAG.

2.2 Struttura semantica e navigazione

Il codice HTML utilizza landmark semantici (`header`, `nav`, `main`, `aside`, `footer`), evitando tag puramente presentazionali e una gerarchia coerente di titoli (`h1-h4`), infatti: le card degli annunci (`cardTemplate.html`) utilizzano `<h3>` per i titoli, evitando coppie `<dt>-<dd>`. In tale modo la gerarchia semantica è preservata: `<h2>` identifica la sezione (es. "Ultimi inseriti"), mentre `<h3>` rappresenta ciascun annuncio.

La navigazione è stata progettata per essere chiara e prevedibile:

- breadcrumb con `aria-current`;
- menu responsive accessibile da tastiera;
- caroselli non ciclici per evitare loop infiniti;

- link sempre descrittivi e mai generici.

2.3 Immagini e testi alternativi

Le immagini above-the-fold usano `loading="eager"`, tutte le altre `lazy`. Sono stati, poi, adottati formati ottimizzati (SVG per il logo, WebP per decorative, JPG per gli annunci) e dimensioni inferiori a 1 MB.

Nel form di pubblicazione l'utente deve scegliere se l'immagine è decorativa o descrittiva e in tal caso il testo alternativo è obbligatorio e limitato a 100 caratteri per garantire sintesi e usabilità con screen reader.

Inoltre, le immagini degli annunci hanno dimensioni definite via HTML per migliorare il rendering.

2.4 Form e messaggi di errore

I form utilizzano `fieldset` e `legend`, con etichette sempre associate ai campi. La validazione è multilivello:

- HTML5 (`required`, `min`, `max`, `step`);
- JavaScript con messaggi `role="alert"` e gestione del focus;
- PHP con la stessa struttura semantica degli errori lato client.

Il focus viene portato automaticamente sul primo campo non valido, con selezione del contenuto alla prima comparsa dell'errore.

2.5 Tipografia e leggibilità

Per garantire una lettura chiara e accessibile sono stati scelti due font distinti:

- **Web:** *Noto Sans*, importato tramite Google Fonts.
- **Stampa:** *Times New Roman*.

La scelta del font web è stata guidata da verifiche sulla leggibilità di caratteri critici (`qp`, `db`, `00`, `iI1l`, `rn`, `m`), sull'altezza della *x* e sull'apertura delle lettere *c* e *a*. Inizialmente era stato considerato *Inter*, poi sostituito da *Noto Sans* per una migliore distinzione di glifi come *ca* e *iI1l* (vedi Figure 1 e 2).

Leggibilità dei prezzi La funzione `PHP Tool::formatPrezzo()` formatta i valori numerici mostrando i decimali solo quando necessario (es. 20€ invece di 20.00€), migliorando la leggibilità.

2.6 Colori, contrasti e accessibilità mobile

Sono stati adottati diversi accorgimenti per garantire una corretta percezione dei contenuti e la riconoscibilità degli elementi interattivi:

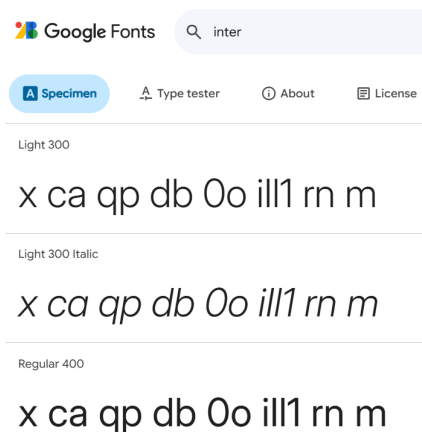


Figura 1: Font: Inter

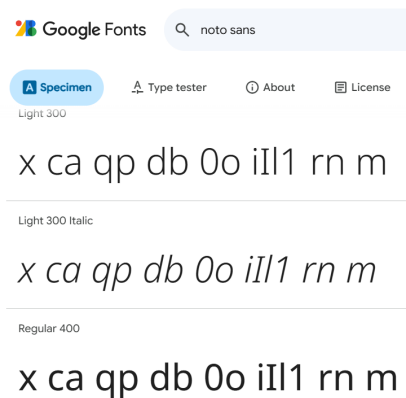


Figura 2: Font: Noto Sans

Link e Call To Action (CTA)

- I link testuali sono sempre sottolineati.
- I link visitati/non visitati nella navbar e nelle card annunci differiscono per colore (contrasto $2.72 \approx 3$) mentre nelle card di categoria il colore del `:visited` è associato al colore della categoria con contrasto ≥ 3 .
- I pulsanti CTA variano colore tra stato visitato e non visitato (in `background-color` e `color`).

([Apri il Sommario dei Contrasti](#) (PDF, 34 KB))

Accessibilità touch su mobile

- Gli elementi cliccabili rispettano dimensioni minime di circa $44\text{px} \times 44\text{px}$.
- Il pannello laterale dei filtri utilizza un overlay e una gestione del focus accessibile (`toggleFiltriAccessibile()`).
- I pulsanti dei caroselli hanno opacità testata e leggibile (`rgba(22, 40, 110, 0.65)`).

3 Validazione, sicurezza e gestione degli errori

3.1 Errori lato client (JavaScript)

La validazione lato client fornisce un feedback immediato e migliora l'esperienza d'uso, ed è attiva solo nei form che dichiarano l'attributo `data-validate`, così da non interferire con i form più semplici. All'avvio della pagina lo script controlla la presenza di eventuali errori generati dal server: se riguardano le immagini il focus viene portato sul primo `input file`, mentre per errori testuali viene selezionato il primo campo non valido; la selezione del contenuto e il trasferimento del focus avvengono solo alla prima comparsa dell'errore per evitare comportamenti troppo restrittivi e bloccanti.

La validazione avviene in tre momenti: al **blur** del campo, durante l'**input** (che rimuove messaggi e ripristina gli attributi ARIA) e al **submit**, quando tutti i campi vengono controllati e l'invio viene eventualmente bloccato. La funzione `validateField()` applica controlli specifici per i campi principali (nome, cognome, email, password, conferma password, città e consenso email), mentre campi come `file`, `alt` e `decorativa` sono esclusi dalla validazione client-side. Gli errori vengono mostrati tramite un contenitore dedicato con elementi dotati di `role="alert"` e con l'applicazione dinamica di `aria-invalid` e `aria-describedby`; la funzione `removeError()` rimuove automaticamente contenitore e attributi non più necessari.

3.2 Errori lato server (PHP)

La validazione lato server garantisce un comportamento corretto anche senza JavaScript, mantenendo coerenza con la validazione client-side. Gli errori riguardano campi generici, immagini (dimensione, formato, testo alternativo) e credenziali di accesso, e vengono restituiti con la stessa struttura semantica dei messaggi lato client, sempre tramite elementi con `role="alert"`. Anche qui vengono applicati dinamicamente `aria-invalid` e `aria-describedby`, e il focus viene portato sul primo campo non valido o sul primo `input file` in caso di errori relativi alle immagini.

Per le immagini vengono controllati dimensione (1 MB), formati consentiti (JPG lato form, JPG/PNG/WEBP lato server) e obbligatorietà del testo alternativo quando l'immagine non è decorativa. Nel login, eventuali errori vengono gestiti tramite messaggi contestuali e tramite il sistema `redirectMessage`, che informa l'utente del motivo del reindirizzamento.

Per la sicurezza degli input viene utilizzata la funzione `strip_tags()`, che rimuove i tag HTML e PHP mantenendo però il testo interno. Questa scelta evita l'inserimento di markup indesiderato senza introdurre messaggi di errore aggiuntivi, mantenendo coerenza tra i vari form: il contenuto viene preservato quando l'utente usa markup con intento semantico, mentre un uso massiccio o sospetto di tag viene neutralizzato assumendo che possa trattarsi di un tentativo malevolo.

4 Testing

4.1 Navigazione da tastiera

È stato controllato l'ordine di tabulazione, la gestione del focus nei pannelli a scomparsa (es. filtri) e l'attivabilità di tutti gli elementi interattivi tramite tastiera. Particolare attenzione è stata dedicata all'`aside` dei filtri, che utilizza `overlay` e `focus trapping`.

4.2 Test con utenti reali

Per garantire che il comportamento del sito fosse chiaro, prevedibile e accessibile, abbiamo svolto una serie di test con utenti esterni al gruppo di sviluppo. Essendo noi i creatori del sistema, avremmo potuto non accorgerci di elementi poco intuitivi; per questo abbiamo coinvolto persone con profili diversi: studenti di varie facoltà, ex studenti, genitori e una persona con mobilità limitata che utilizza una sola mano. Questo ci ha permesso di osservare l'interazione da prospettive differenti e di verificare che l'esperienza d'uso rimanesse comprensibile e coerente per utenti con esigenze e competenze eterogenee.